Beyond the Syllabus

Randomness, Refined: Karger–Stein, Derandomization, and the Limits of Coin-Flipping

Week 15 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why look beyond the syllabus?

- This week you met five randomized algorithms and the Las Vegas/Monte Carlo distinction: QuickSort ($O(n \log n)$ expected), Quickselect ($O(n)$ expected), Karger's contraction algorithm for global min-cut, reservoir sampling, and universal hashing / Miller–Rabin – all analyzed with indicator variables, linearity of expectation, and the union bound.
- Three questions a curious student should ask next:
  1. Karger's algorithm, as amplified in lecture, reruns from scratch $\Theta(n^2 \log n)$ times. Can the *same* probability argument you proved be reused more cleverly instead of just repeated?
  2. Is randomness a genuinely **separate** computational resource from determinism – or can every efficient randomized algorithm, in principle, be **derandomized** at only polynomial cost?
  3. For Quickselect specifically, has that derandomization question already been *settled*?

How this supplement is different from a survey

Every section below walks through the *actual mechanics* of a result – reusing a proof technique you already hand-traced this week – not just a headline number. Every specific bound is sourced to a specific paper.

Recap: the cost of amplifying Karger's algorithm

What you proved this week

A single contraction run keeps a fixed min-cut with probability $\geq 1/\binom{n}{2}$ (Theorem, lecture notes). Amplifying by $T = \binom{n}{2} \ln n = \Theta(n^2 \log n)$ **independent, from-scratch** reruns drives the failure probability down to $\leq 1/n$.

- Each full run costs $\Theta(m)$ (one pass of $n-2$ edge contractions on a graph with m edges), so the amplified scheme costs $\Theta(mn^2 \log n)$ total.
- That is polynomial – but it treats every rerun as throwing away *all* prior work and starting over from n vertices. Is that wasteful?

The key idea (Karger & Stein, JACM 1996): recurse, don't restart

Karger & Stein, "A New Approach to the Minimum Cut Problem," *J. ACM* 43(4):601–640, 1996

Reuse the *exact same* telescoping survival-probability argument from this week's lecture – but stop it early, and recurse.

- The lecture's proof shows: contracting from n down to n' vertices keeps a fixed min-cut with probability $\geq \dfrac{n'(n'-1)}{n(n-1)}$ (same telescoping product, just not carried all the way to 2).
- Set $n' = n/\sqrt{2}$: that ratio is $\approx 1/2$. So contracting *halfway* (in this $\sqrt{2}$ sense) survives with probability $\geq 1/2$ – much better odds than surviving all the way to 2 vertices.
- **Recursive algorithm:** contract $n \to n/\sqrt{2}$ once, then recurse **twice**, independently, on that smaller graph, and keep the better of the two results. Base case: a small constant-size graph.

Solving the two recurrences

Time

$T(n) = 2\,T(n/\sqrt{2}) + O(n^2)$ (the $O(n^2)$ is the cost of one contraction phase down to $n/\sqrt{2}$ vertices). Since $\log_{\sqrt{2}} 2 = 2$, the branching exactly balances the shrink rate at every level, giving

$$T(n) = O(n^2 \log n).$$

Success probability

$P(n) = 1 - \left(1 - \frac{1}{2}P(n/\sqrt{2})\right)^2$ (fails only if *both* recursive branches fail). Solving this recurrence gives

$$P(n) = \Theta(1/\log n).$$

The payoff

Plain Karger: one run succeeds with probability $\Theta(1/n^2)$. One Karger–Stein call succeeds with probability $\Theta(1/\log n)$ – exponentially better per run, for only an extra $\log n$ factor in the per-run cost.

Putting it together: amplifying the better algorithm

- Exactly the union-bound amplification argument from this week's Corollary (Karger amplification) and Theorem (contention resolution) applies again: to drive failure probability down to $\leq 1/n$ with a per-run success probability of $\Theta(1/\log n)$, you need only $O(\log n)$ independent reruns instead of $O(n^2 \log n)$.
- Total cost: $O(\log n)$ reruns $\times$ $O(n^2 \log n)$ per run $= O(n^2 \log^3 n)$ – exactly the bound Karger and Stein proved.

Scheme	Total time	Idea
Plain Karger, amplified (lecture)	$\Theta(mn^2 \log n)$	$\Theta(n^2 \log n)$ independent full reruns
Karger–Stein (1996)	$O(n^2 \log^3 n)$	recurse on 2 halved copies, reuse the same proof

For sparse graphs ($m \approx n$) this removes an entire factor of n – no new probability idea, just reusing the lecture's own telescoping argument *locally* instead of only globally.

Is Karger–Stein itself optimal?

Gupta, Lee, and Li, "The Karger–Stein Algorithm Is Optimal for k-Cut," STOC 2020

The same recursive-contraction strategy, applied to the more general problem of splitting a graph into k pieces (not just 2), outputs any fixed minimum k-cut with probability that matches the best possible bound, up to lower-order terms.

- Same theme as Week 3's "is Union-Find provably optimal" story: a simple, 20-line randomized idea from the 1990s turns out to be not just good, but (nearly) the best *any* algorithm of its kind can do – confirmed 24 years later, and for a strictly harder generalization of the problem.

The question: is BPP = P?

- **BPP** is the class of problems solvable by a Monte Carlo algorithm running in polynomial time with bounded error (exactly this week's Monte Carlo definition, formalized as a complexity class).
- Open since the 1980s: is **every** such algorithm derandomizable – replaceable by a deterministic polynomial-time algorithm? I.e., is $BPP = P$?
- The classical attack, **hardness-vs-randomness** (Nisan & Wigderson, *J. Comput. Syst. Sci.* 1994): *if* some function needs large circuits to compute in the worst case, *then* that same hardness can be converted into a pseudorandom generator (PRG) – a short deterministic seed that "looks random" enough to fool any small circuit, replacing true coin flips at only polynomial cost.

Why this matters here

If this program succeeds completely, every algorithm in this week's lecture – QuickSort, Quickselect, Karger's contraction – would have a deterministic polynomial-time counterpart with (asymptotically) the same running time. That is a long-standing *research program*, not a settled fact.

A landmark tightening (2020/2022)

Doron, Moshkovitz, Oh, and Zuckerman, "Nearly Optimal Pseudorandomness from Hardness," FOCS 2020 / *J. ACM* 69(6), Article 43, 2022 – *DOI:* 10.1145/3555307

Construct pseudorandom generators whose parameters (how hard the seed function must be vs. how large a circuit the generator fools) are **nearly optimal** for the hardness-vs-randomness paradigm – closing most of a 26-year-old gap between what Nisan–Wigderson-style constructions could prove and what the paradigm should, in principle, be able to deliver.

- This does not resolve $BPP \stackrel{?}{=} P$ (it is still conditional on an unproven worst-case hardness assumption) – but it means *if* that assumption holds, the derandomization it buys you is essentially as good as this framework can ever give.

Still being tightened – this month

Doron, Moshkovitz, Oh, and Zuckerman, ECCC Technical Report TR26-082, May 2026

The same four authors continue optimizing exactly this tradeoff: how much worst-case circuit hardness you must assume vs. how large a circuit the resulting generator can fool, pushing the two parameters closer together than the 2022 result.

An open program, moving in real time

As of this month (August 2026), the parameters of a 30-year-old paradigm – hardness-vs-randomness – are still being sharpened. This is not settled textbook history; it is active research you could, in principle, read the newest preprint of.

One case where derandomization already won – in 1973

- The *general* $BPP \stackrel{?}{=} P$ question is open, but individual problems can be – and have been – derandomized completely.
- This week's own **Quickselect** (expected $O(n)$, worst-case $\Theta(n^2)$) has a fully deterministic worst-case-$O(n)$ counterpart, found *47 years before* the 2020 pseudorandomness result above:

Blum, Floyd, Pratt, Rivest, and Tarjan, "Time Bounds for Selection," *J. Comput. Syst. Sci.* 7:448–461, 1973

The **median-of-medians** algorithm: split into groups of 5, recursively find the median of each group's median, and use *that* as a provably good pivot – no coin flips anywhere, worst-case $O(n)$, guaranteed on every input.

The lesson

"Open in general" does not mean "open for the problem you care about." Median-of-medians is asymptotically optimal but has large constants, which is exactly why randomized Quickselect – simpler, faster in practice – is what you actually traced this week.

The current fastest max-flow algorithm is randomized

Chen, Kyng, Liu, Peng, Probst Gutenberg, and Sachdeva, "Maximum Flow and Minimum-Cost Flow in Almost-Linear Time," FOCS 2022 (arXiv:2203.00671)

Computes exact maximum flow and minimum-cost flow on graphs with m edges and polynomially bounded integer data in $m^{1+o(1)}$ time – a landmark speedup over the polynomial (but far slower) flow algorithms in a typical course.

- The algorithm's numerical core relies on **randomized** graph sparsification and randomized linear-algebra solvers – it is not just historically related to this week's ideas, it is *built from* the same coin-flipping toolkit.
- As of August 2026, no deterministic algorithm matches this speed. Randomness is not a classroom curiosity here; it is the current frontier.

Summary

1. **Even the algorithm you hand-traced had headroom left:** Karger–Stein (1996) reuses this week's own telescoping-probability proof *locally* instead of only globally, cutting an amplified $\Theta(mn^2 \log n)$ scheme down to $O(n^2 \log^3 n)$ – and that recursive strategy was proven (near-)optimal for the general k-cut problem in 2020.
2. **Is randomness a separate resource from determinism?** The hardness-vs-randomness program (Nisan–Wigderson, 1994) is still being tightened – most recently this month (Doron, Moshkovitz, Oh, Zuckerman, May 2026) – with no final answer yet.
3. **But some cases are already closed:** Quickselect's own worst case was fully derandomized in 1973 (median-of-medians), decades before the general question was even well-posed.
4. **Randomization is still winning at the frontier:** the fastest known max-flow algorithm (2022) is randomized at its numerical core, with nothing deterministic matching it as of today.

Looking back over the course

The lecture's own closing section traces six paradigms across fifteen weeks. This supplement's point: several of them – greedy's exchange argument, flow's augmenting paths, this week's contraction algorithm – were sharpened within the last five years by exactly the move Karger–Stein made: take a proof you trust, and reuse it more cleverly.